```
$csvRow4 = 'binary' . "\x00" . 'safe_field'
    . ',we"ird"_fi"eld"_1' // multiple enclosure chars mid-data,
                           not wrapped
    . ',"we"ird"_fi"eld"_2"' // multiple enclosure chars wrapped between
                             RFC-compliant enclosures
    . ',"we\"ird\"_fi\"eld\"_3"' // slash-escaped enclosures between RFC-
                                 compliant enclosures
;
print_r(str_getcsv($csvRow4, escape: '\\'));
```

```
Output:
Array
(
    [0] => binarysafe_field
    [1] => we"ird"_fi"eld"_1
    [2] => weird"_fi"eld"_2"
    [3] => we\"ird\"_fi\"eld\"_3
)
```

From above, we can deduce a few things. First, `str_getcsv()` is binary safe, like most other core string functions: we get `binarysafe_field` as our first string despite the null byte in the middle. Second, the enclosure character can be used freely in a data field provided the field isn't enclosed but behaves strangely if the field is enclosed: we can see from `weird_field_2` that one of the double-quotes has simply vanished, and we get an extra double-quote character at the end of the field data. Finally, if we use the slash escape mechanism, the double-quotes that are liberally spread around the field data are handled correctly, but we get the escape character as part of the output. I suppose the moral of this story is to always use the RFC enclosure and escape mechanisms and to thoroughly test your code and check the input data. `str_getcsv()` does accept additional arguments, allowing you to define your own enclosure and escape characters, but your mileage may vary: the standard/conventional ways are far more portable.

Another thing to bear in mind: `str_getcsv()` only deals with *single* lines. There are many posts and blogs out there on the theme of "PHP's `str_getcsv()` function is broken" that are often problems in file handling. The full RFC 4180 specification stipulates how to handle multi-line CSV data, which becomes important when field